

# Методичка 9. Процедуры и функции

## 1. Зачем нужна эта тема

---

Функции нужны, чтобы не писать один и тот же код много раз и делить программу на понятные части.

Когда программа маленькая, весь код можно написать сверху вниз. Но когда программа растёт, в ней становится сложно ориентироваться.

Например, в игре могут быть отдельные действия:

- вывести поле;
- сделать ход;
- проверить победу;
- вывести меню;
- получить число от пользователя;
- проверить правильность ввода.

Если всё написать в одном месте, код быстро станет длинным и запутанным.

Функции позволяют дать отдельному действию имя и вызывать его тогда, когда нужно.

Эта тема связана со всеми предыдущими:

- внутри функций можно использовать переменные;
- условия;
- циклы;
- списки;
- строки;
- срезы.

## 2. Главная идея темы

---

Функция — это отдельный блок кода с именем.

Функцию можно вызвать несколько раз.

Пример:

```
def say_hello():  
    print("Привет")  
  
say_hello()  
say_hello()
```

Функция `say_hello()` описана один раз, но вызвана два раза.

**Функция** — именованный блок кода, который можно запускать.

**Вызов функции** — команда, которая запускает функцию.

**Параметр** — переменная внутри функции, через которую функция получает данные.

**Аргумент** — конкретное значение, которое передают в функцию при вызове.

**return** — команда, которая возвращает результат из функции.

**Процедура** — функция, которая ничего не возвращает, а просто выполняет действие. В Python технически всё равно используется `def`.

## 3. Базовый синтаксис

---

### Функция без параметров

```
def print_menu():  
    print("1 — Начать")  
    print("2 — Настройки")  
    print("0 — Выход")  
  
print_menu()
```

Что происходит:

- `def print_menu():` создаёт функцию;
- строки с отступом — тело функции;
- `print_menu()` вызывает функцию.

На что обратить внимание:

- после имени функции ставятся круглые скобки;
- после строки `def` ставится двоеточие;
- тело функции пишется с отступом;

- функция не выполнится, пока её не вызовут.

## Функция с параметром

```
def greet(name):  
    print("Привет,", name)  
  
greet("Аня")  
greet("Борис")
```

`name` — параметр. `"Аня"` и `"Борис"` — аргументы.

## Функция с несколькими параметрами

```
def print_sum(a, b):  
    print(a + b)  
  
print_sum(5, 7)
```

Функция получает два числа и выводит их сумму.

## Функция с return

```
def get_sum(a, b):  
    return a + b  
  
result = get_sum(5, 7)  
  
print(result)
```

`return` возвращает результат туда, где функция была вызвана.

## Процедура и функция

Процедура выполняет действие:

```
def print_error():  
    print("Ошибка ввода")
```

Функция возвращает результат:

```
def is_even(number):  
    return number % 2 == 0
```

## 4. Разбор темы от простого к сложному

---

### Шаг 1. Самая простая функция

```
def say_hello():  
    print("Привет")  
  
say_hello()
```

Функция просто выводит сообщение.

### Шаг 2. Функция с параметром

```
def say_hello(name):  
    print("Привет,", name)  
  
say_hello("Маша")  
say_hello("Петя")
```

Одна функция работает с разными значениями.

### Шаг 3. Функция для вычисления

```
def square(number):  
    return number * number  
  
result = square(5)  
  
print(result)
```

Функция возвращает квадрат числа.

### Шаг 4. Функция для проверки

```
def is_even(number):  
    return number % 2 == 0  
  
number = int(input("Введите число: "))  
  
if is_even(number):  
    print("Чётное")  
else:  
    print("Нечётное")
```

Функция возвращает `True` или `False` .

## Шаг 5. Функция для ввода

```
def input_number(message):  
    number = int(input(message))  
    return number  
  
age = input_number("Введите возраст: ")  
  
print("Возраст:", age)
```

Функция помогает не повторять одинаковый ввод чисел.

## Шаг 6. Функция для работы со строкой

```
def count_letter(text, target):  
    count = 0  
  
    for letter in text:  
        if letter == target:  
            count = count + 1  
  
    return count  
  
word = input("Введите слово: ")  
  
result = count_letter(word, "a")  
  
print("Букв a:", result)
```

Функция получает строку и букву, а возвращает количество таких букв.

## Шаг 7. Функция для списка

```
def get_sum(numbers):
    total = 0

    for number in numbers:
        total = total + number

    return total

items = [10, 20, 30]

print(get_sum(items))
```

Функция работает со списком чисел.

## Шаг 8. Разделение программы на функции

```
def print_menu():
    print("1 – Сказать привет")
    print("0 – Выход")

def handle_command(command):
    if command == "1":
        print("Привет")
    elif command == "0":
        print("Выход")
    else:
        print("Неизвестная команда")

print_menu()
user_command = input("Введите команду: ")
handle_command(user_command)
```

Программа становится понятнее, потому что каждое действие имеет имя.

## 5. Частые ошибки учеников

---

### Ошибка 1. Описали функцию, но не вызвали

Неправильно:

```
def say_hello():
    print("Привет")
```

Почему это ошибка:

Функция создана, но не запущена. Поэтому на экран ничего не выведется.

Правильно:

```
def say_hello():  
    print("Привет")  
  
say_hello()
```

## Ошибка 2. Забыли круглые скобки при вызове

Неправильно:

```
def say_hello():  
    print("Привет")  
  
say_hello
```

Почему это ошибка:

Без круглых скобок функция не вызывается.

Правильно:

```
def say_hello():  
    print("Привет")  
  
say_hello()
```

## Ошибка 3. Забыли двоеточие после def

Неправильно:

```
def say_hello()  
    print("Привет")
```

Почему это ошибка:

После объявления функции нужно ставить двоеточие.

Правильно:

```
def say_hello():  
    print("Привет")
```

## Ошибка 4. Забыли отступ в теле функции

Неправильно:

```
def say_hello():  
print("Привет")
```

Почему это ошибка:

Python понимает тело функции по отступу.

Правильно:

```
def say_hello():  
    print("Привет")
```

## Ошибка 5. Путают print и return

Неправильно:

```
def get_sum(a, b):  
    print(a + b)  
  
result = get_sum(5, 7)  
  
print(result)
```

Почему это ошибка:

`print()` только выводит на экран. Но функция ничего не возвращает, поэтому в `result` попадёт `None`.

Правильно:

```
def get_sum(a, b):  
    return a + b  
  
result = get_sum(5, 7)  
  
print(result)
```



## Ошибка 6. Пишут код после return внутри функции

Неправильно:

```
def get_sum(a, b):  
    return a + b  
    print("Готово")
```

Почему это ошибка:

После `return` функция сразу завершается. Строка `print("Готово")` не выполнится.

Правильно:

```
def get_sum(a, b):  
    result = a + b  
    return result
```

## Ошибка 7. Используют переменную функции снаружи

Неправильно:

```
def create_number():  
    number = 10  
  
create_number()  
  
print(number)
```

Почему это ошибка:

Переменная `number` создана внутри функции и доступна только внутри неё.

Правильно:

```
def create_number():  
    number = 10  
    return number  
  
result = create_number()  
  
print(result)
```

## Ошибка 8. Слишком большая функция

Неправильно:

```
def program():
    print("Меню")
    name = input("Имя: ")
    age = int(input("Возраст: "))
    if age >= 18:
        print("Можно")
    else:
        print("Нельзя")
    print("Конец")
```

Почему это плохо:

Функция делает слишком много разных действий. Её сложнее читать и проверять.

Лучше разделить:

```
def input_age():
    return int(input("Возраст: "))

def is_adult(age):
    return age >= 18

age = input_age()

if is_adult(age):
    print("Можно")
else:
    print("Нельзя")
```

## 6. Мини-примеры для закрепления

---

### Пример 1. Приветствие

```
def hello():
    print("Привет")

hello()
```

Закрепляется создание и вызов функции.

### Пример 2. Приветствие по имени

```
def hello(name):  
    print("Привет,", name)  
  
hello("Аня")
```

Закрепляется параметр функции.

### Пример 3. Сумма двух чисел

```
def get_sum(a, b):  
    return a + b  
  
print(get_sum(3, 4))
```

Закрепляется `return`.

### Пример 4. Проверка чётности

```
def is_even(number):  
    return number % 2 == 0  
  
print(is_even(10))
```

Закрепляется функция, которая возвращает логическое значение.

### Пример 5. Максимум из двух чисел

```
def get_max(a, b):  
    if a > b:  
        return a  
    else:  
        return b  
  
print(get_max(10, 7))
```

Закрепляется `return` внутри условия.

### Пример 6. Подсчёт букв

```
def count_letter(text, target):
    count = 0

    for letter in text:
        if letter == target:
            count = count + 1

    return count

print(count_letter("banana", "a"))
```

Закрепляется функция со строкой и циклом.

## Пример 7. Сумма списка

```
def get_sum(numbers):
    total = 0

    for number in numbers:
        total = total + number

    return total

print(get_sum([1, 2, 3]))
```

Закрепляется передача списка в функцию.

## Пример 8. Вывод меню

```
def print_menu():
    print("1 — Начать")
    print("0 — Выход")

print_menu()
```

Закрепляется процедура без `return`.

# 7. Практические задания

---

## Лёгкий уровень

1. Написать функцию `hello()`, которая выводит `"Привет"`.
2. Написать функцию `hello_name(name)`, которая приветствует пользователя по имени.

3. Написать функцию `get_sum(a, b)`, которая возвращает сумму двух чисел.
4. Написать функцию `get_square(number)`, которая возвращает квадрат числа.
5. Написать функцию `is_even(number)`, которая возвращает `True`, если число чётное.

## Средний уровень

1. Написать функцию, которая принимает список чисел и возвращает их сумму.
2. Написать функцию, которая принимает строку и букву, а возвращает количество таких букв в строке.
3. Написать функцию, которая принимает два числа и возвращает большее.
4. Написать функцию, которая принимает возраст и возвращает `True`, если пользователь совершеннолетний.
5. Написать функцию для безопасного вывода меню программы.

## Повышенный уровень

1. Написать функцию, которая принимает список чисел и возвращает количество положительных чисел.
2. Написать функцию, которая принимает строку и возвращает её без первого и последнего символа.
3. Написать программу с меню, где вывод меню, обработка команды и ввод команды вынесены в отдельные функции.

## 8. Контрольные вопросы

---

1. Что такое функция?
2. Для чего нужны функции?
3. Что делает ключевое слово `def`?
4. Что такое вызов функции?
5. Что такое параметр?
6. Что такое аргумент?
7. Чем отличается параметр от аргумента?
8. Что делает `return`?
9. Чем `print` отличается от `return`?
10. Что будет, если описать функцию, но не вызвать её?
11. Почему большие программы лучше делить на функции?
12. Можно ли передать список в функцию?

## 9. Краткий конспект в конце

---

Главная идея темы: функции помогают делить программу на понятные части и не повторять код.

Основные конструкции:

```
def hello():  
    print("Привет")
```

```
hello()
```

```
def get_sum(a, b):  
    return a + b
```

```
result = get_sum(5, 7)
```

Важно запомнить:

- функция создаётся через `def` ;
- функция не выполняется, пока её не вызовут;
- параметры пишутся в скобках;
- аргументы передаются при вызове;
- `return` возвращает результат;
- `print` только выводит на экран;
- функции делают код чище и понятнее.

Частые ошибки:

- забыть вызвать функцию;
- забыть круглые скобки;
- забыть двоеточие;
- перепутать `print` и `return` ;
- использовать внутреннюю переменную функции снаружи;
- делать одну огромную функцию.

## 10. Что ученик должен уметь после методички

---

После этой темы ученик должен уметь:

- создавать функции;
- вызывать функции;

- использовать параметры;
- передавать аргументы;
- возвращать результат через `return` ;
- отличать `print` от `return` ;
- писать функции для чисел, строк и списков;
- разбивать простую программу на несколько функций;
- находить типичные ошибки в функциях.